Vector Processor

Vector Processor

Vector: A vector is simply an ordered collection of numbers.

Example: V = [5, 10, 15, 20]

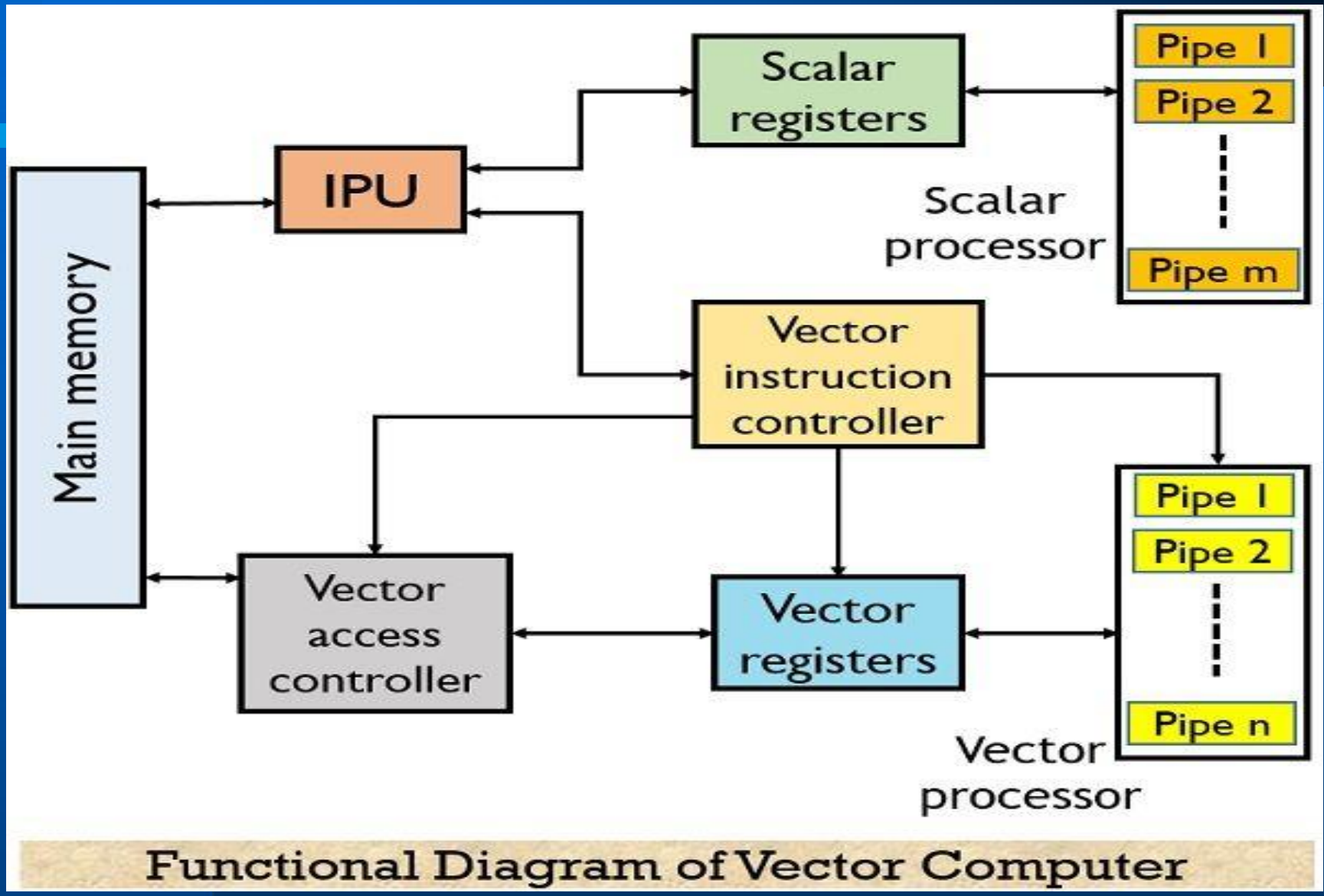
A **Vector Processor** is a CPU designed to execute **one instruction on multiple data elements** simultaneously. Instead of processing one number at a time, it processes an entire array (vector) using a single instruction.

Suppose: A = [2,4,6,8], B = [1,3,5,7]

Addition: C = A + B

Result: C = [3,7,11,15]

Basic Architecture



Functional Diagram of Vector Computer

Vector Processor Components

Main Memory: Stores vectors.

Example: A[1000] B[1000] C[1000]

Vector Registers: Large registers that store an entire vector.

Example: VR1, VR2, VR3

Each may hold: 64 elements, 128 elements, 256 elements

VR1 [2 4 6 8]

VR2 [1 3 5 7]

Addition

VR3 [3 7 11 15]

Functional Units: Perform operations.

Examples: Addition, Multiplication, Division, Logical operations

Control Unit: Issues vector instructions.

Example: VADD, VMUL, VDIV, VLOAD, VSTORE

Vector Processor Components

Vector Instructions

VLOAD, VSTORE, VADD, VSUB, VMUL, VDIV, VCMP

Example

VLOAD VR1, A -> Load vector A

VADD VR3, VR1, VR2 -> Add vectors

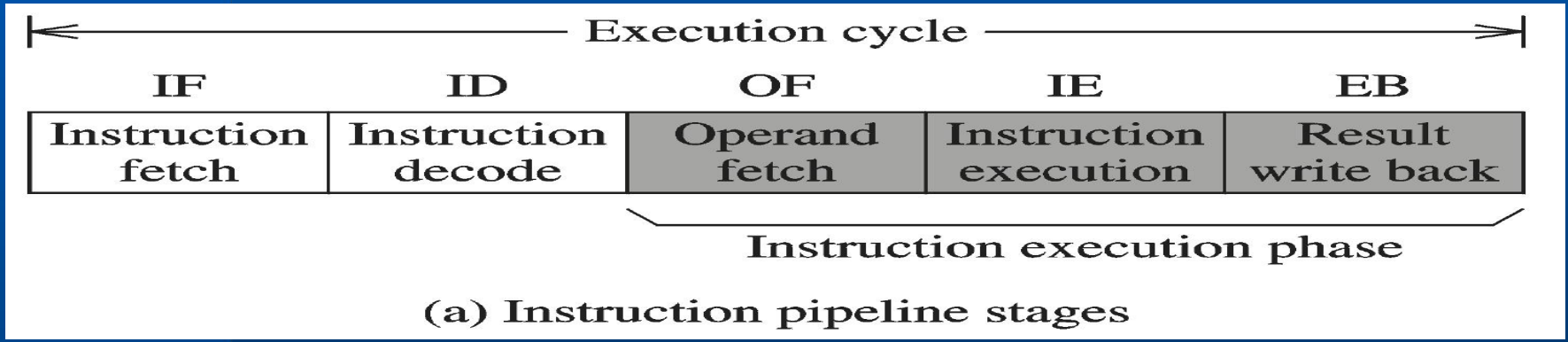
VSTORE C, VR3 -> Store result

Vector Processor Components

Vector Pipeline: A hardware organization where different elements of a vector are processed simultaneously in different pipeline stages, allowing one result to be produced every clock cycle after the pipeline is filled.

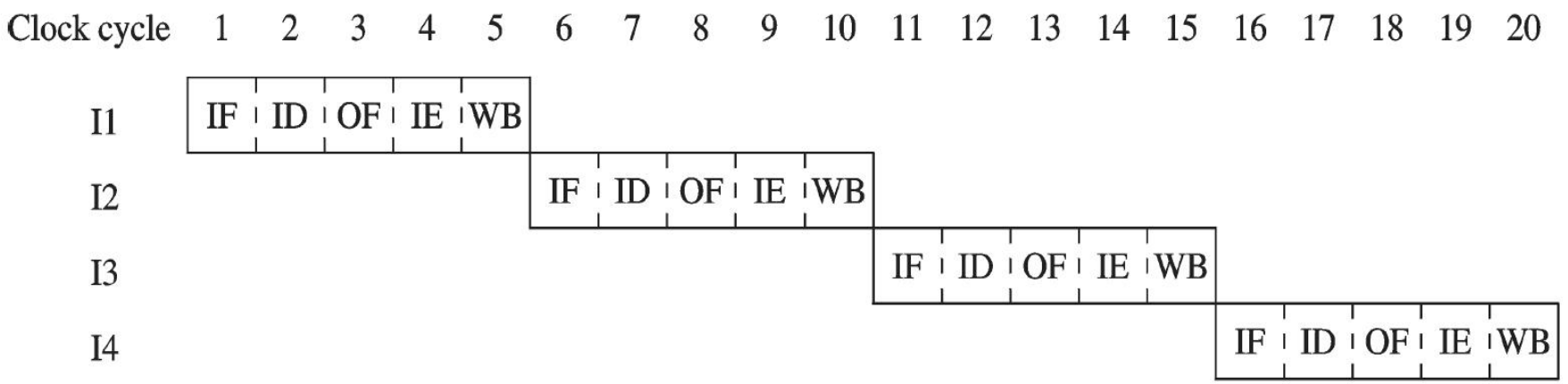
Combines

- Vector processing — one operation on an entire array
- Instruction pipelining — multiple instructions are overlapped

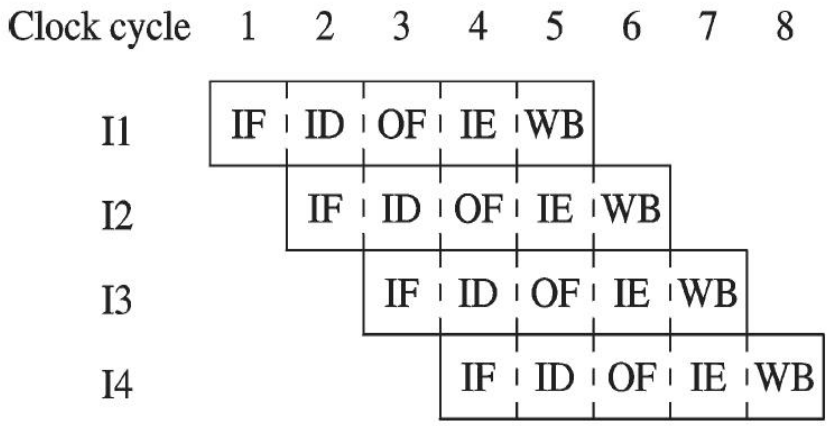


(a) Instruction pipeline stages

Vector Pipeline



(a) Serial execution



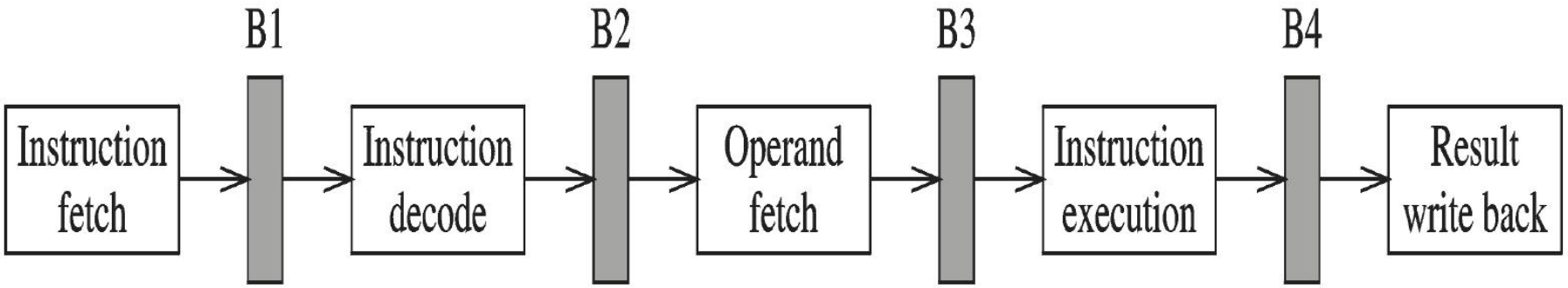
Serial execution: 20 cycles

Pipelined execution: 8 cycles

(b) Pipelined execution

Vector Pipeline

- Pipelining requires buffers
  - Each buffer holds a single value
  - Uses just-in-time principle
    - » Any delay in one stage affects the entire pipeline flow
  - Ideal scenario: equal work for each stage
    - » Sometimes it is not possible
    - » Slowest stage determines the flow rate in the entire pipeline



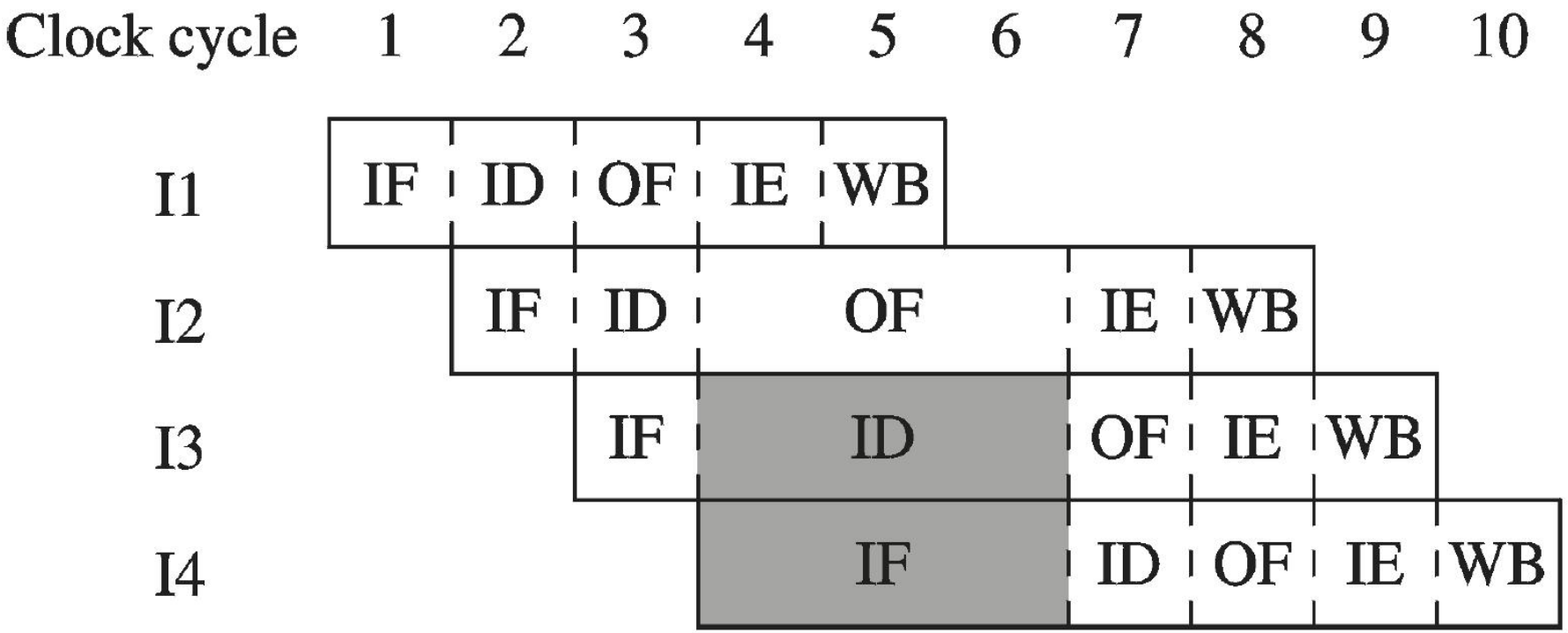
Vector Pipeline

Some reasons for unequal work stages

* A complex step cannot be subdivided conveniently
* An operation takes variable amount of time to execute
  - » EX: Operand fetch time depends on where the operands are located
    - – Registers
    - – Cache
    - – Memory
* Complexity of operation depends on the type of operation
  - » Add: may take one cycle
  - » Multiply: may take several cycles

Vector Pipeline

- Operand fetch of I2 takes three cycles
  - Pipeline ***stalls*** for two cycles
    - » Caused by hazards
  - Pipeline stalls reduce overall throughput



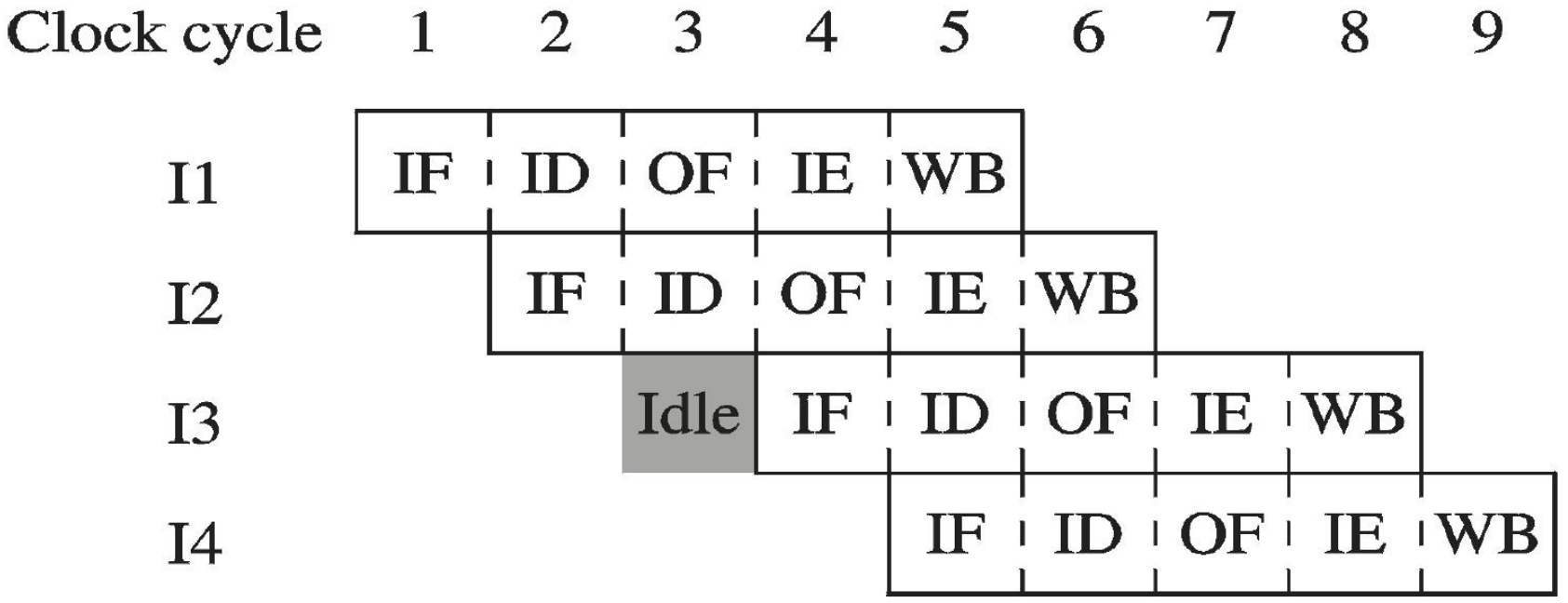
Vector Pipeline

Three types of hazards

* Resource hazards
  - » Occurs when two or more instructions use the same resource
  - » Also called ***structural hazards***
* Data hazards
  - » Caused by data dependencies between instructions
    - – Example: Result produced by I1 is read by I2
* Control hazards
  - » Default: sequential execution suits pipelining
  - » Altering control flow (e.g., branching) causes problems
    - – Introduce control dependencies

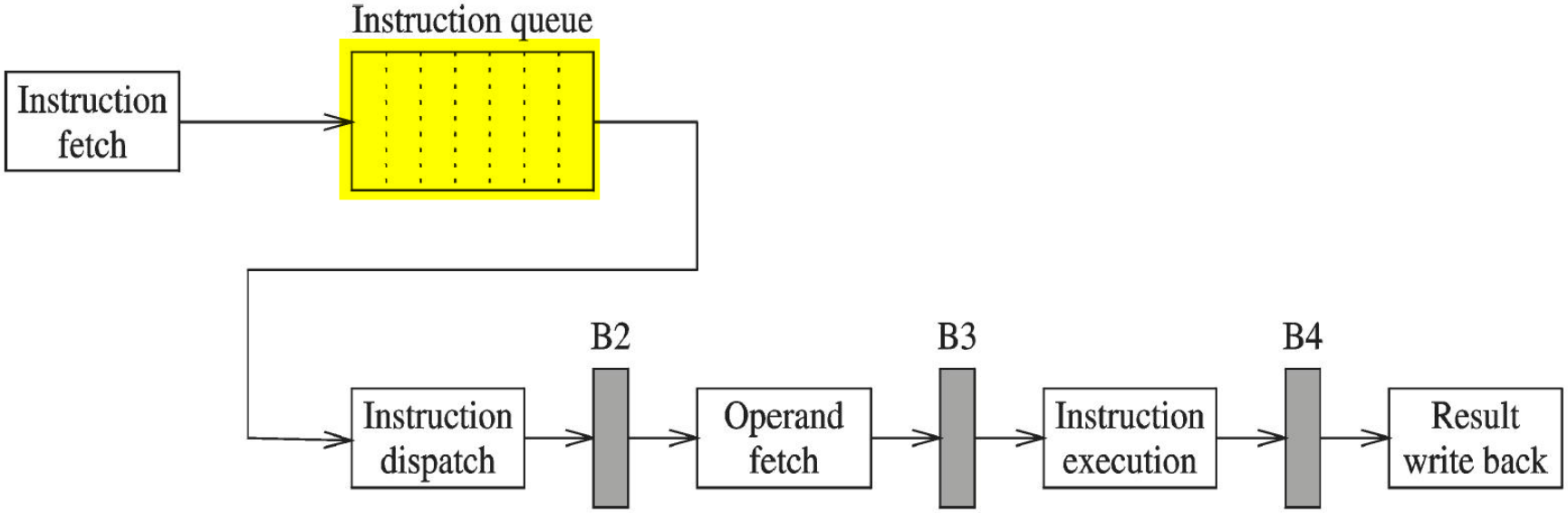
Handling Resource Conflicts

- Example
  - * Conflict for memory in clock cycle 3
    - » I1 fetches operand
    - » I3 delays its instruction fetch from the same memory



Handling Resource Conflicts

- Minimizing the impact of resource conflicts
  - * Increase available resources
  - * Prefetch
    - » Relaxes just-in-time principle
    - » Example: Instruction queue

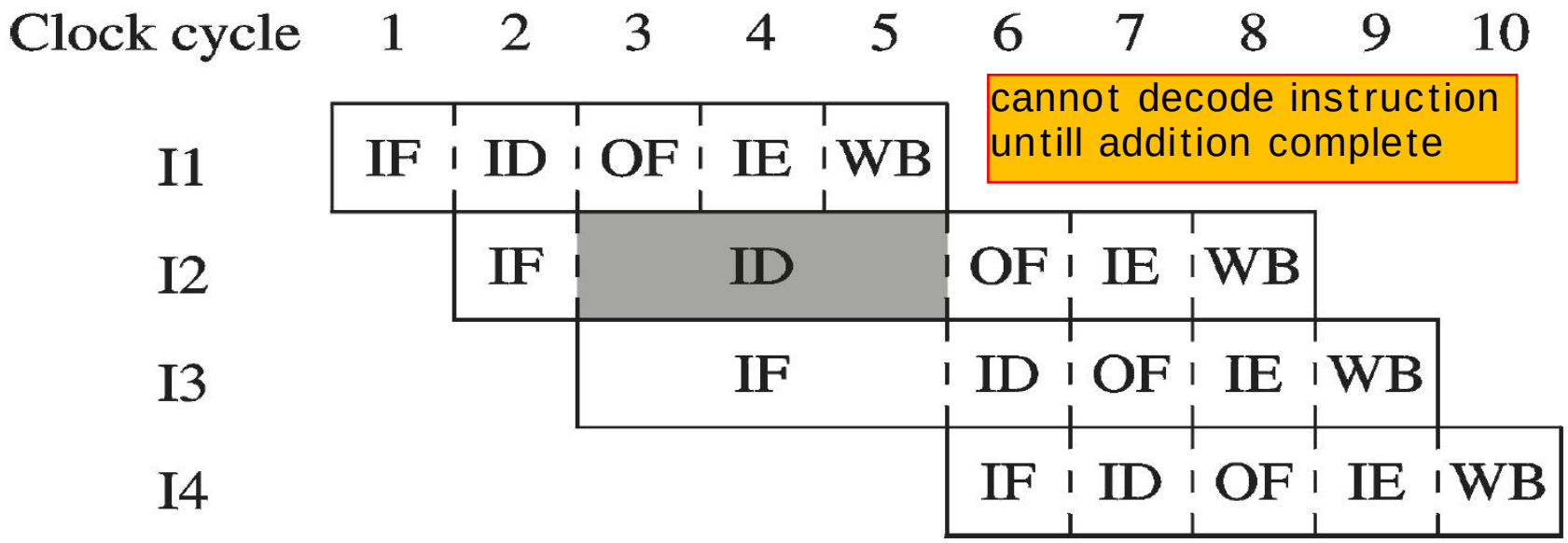


Data Hazards

- Example

```
I1: add   R2,R3,R4   /* R2 = R3 + R4 */
I2: sub   R5,R6,R2   /* R5 = R6 – R2 */
```

- Introduces data dependency between I1 and I2



Data Hazards

Problems

- Three types of data dependencies require attention
  - * Read-After-Write (RAW)
    - » One instruction writes that is later read by the other instruction
  - * Write-After-Read (WAR)
    - » One instruction reads from register/memory that is later written by the other instruction
  - * Write-After-Write (WAW)
    - » One instruction writes into register/memory that is later written by the other instruction

---

- * Read-After-Read (RAR)
  - » No conflict

Data Hazards

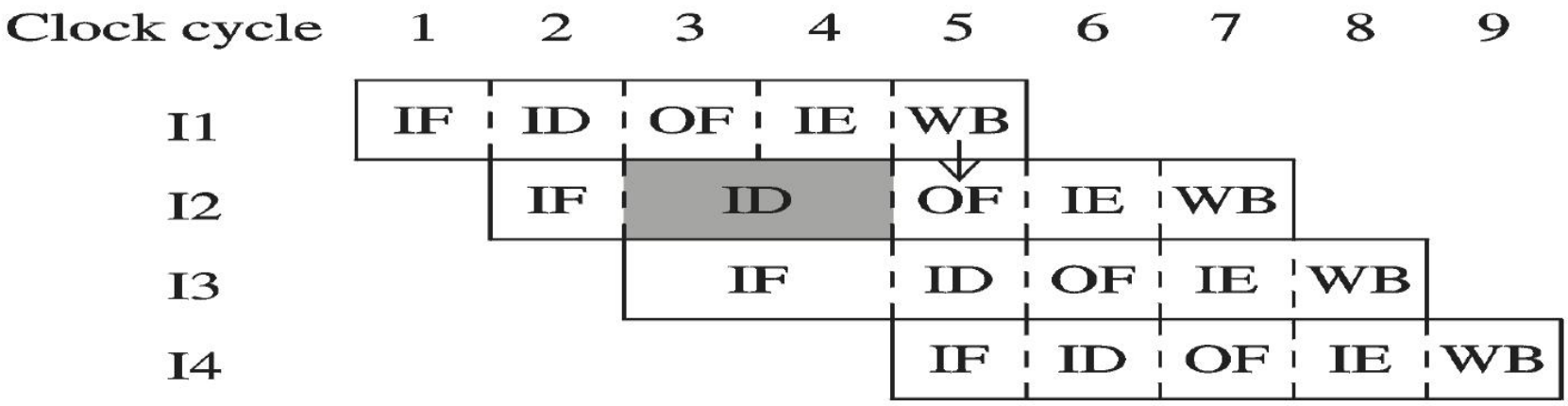
- Data dependencies have two implications
  - * Correctness issue
    - » Detect dependency and stall
      - – We have to stall the SUB instruction
  - * Efficiency issue
    - » Try to minimize pipeline stalls
- Two techniques to handle data dependencies
  - * Register interlocking
    - » Also called ***bypassing***
  - * Register forwarding
    - » General technique

Instead of waiting to save a result in the register file, the processor grabs it directly from the internal wires and sends it straight to the next instruction that needs it.

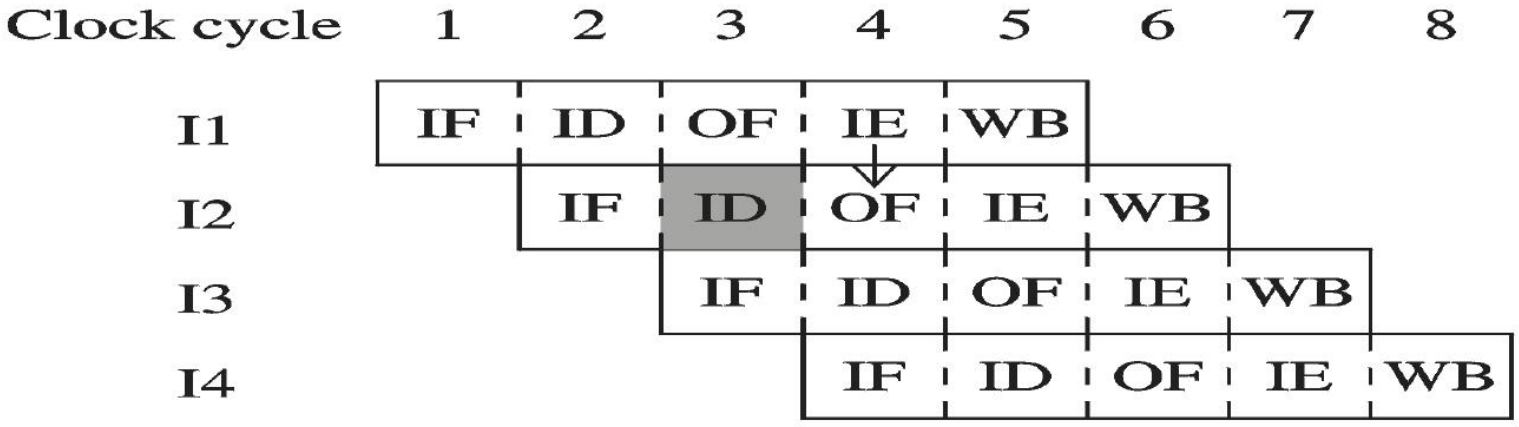
Data Hazards

- Register interlocking
  - Provide output result as soon as possible
- An Example
  - Forward 1 scheme
    - Output of I1 is given to I2 as we write the result into destination register of I1
    - Reduces pipeline stall by one cycle
  - Forward 2 scheme
    - Output of I1 is given to I2 during the IE stage of I1
    - Reduces pipeline stall by two cycles

Data Hazards



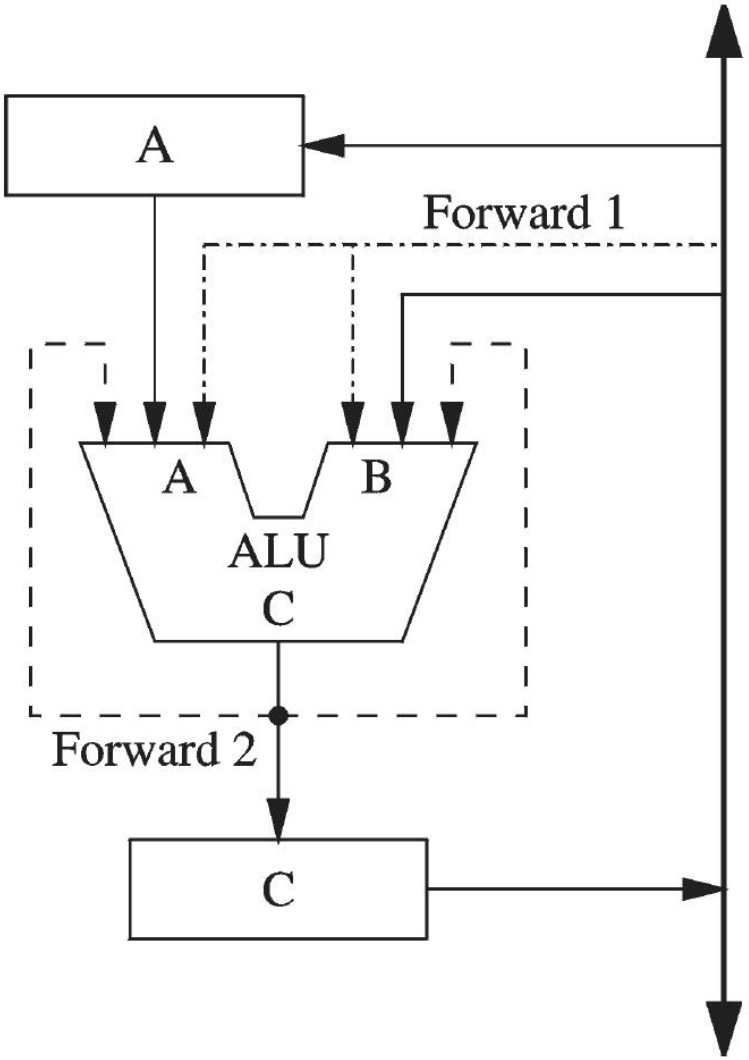
(a) Forward scheme 1



(b) Forward scheme 2

Data Hazards

- Implementation of forwarding in hardware
  - Forward 1 scheme
    - » Result is given as input from the bus
      - – Not from A
  - Forward 2 scheme
    - » Result is given as input from the ALU output



Data Hazards

- Register interlocking
  - Associate a bit with each register
    - Indicates whether the contents are correct
      - 0 : contents can be used
      - 1 : do not use contents

Vector Pipeline

Vector Chaining: It allows the output of one vector instruction to become the input of another **without waiting for the entire vector to finish**.

- **Example:** VMUL -> VADD -> VSTORE

As soon as the first multiplication result is ready, addition begins immediately.

Vector Length Register (VLR): Determines how many elements participate in an operation.

Vector Length = 64

Only first 64 elements are processed.

Vector Pipeline

Vector Mask Register: Sometimes only selected elements are processed.

Example: Vector -> [10 20 30 40]

Mask: [1 0 1 0]
Result

- 10 processed
- 20 skipped
- 30 processed
- 40 skipped

Useful in conditional execution.

Vector Pipeline

Advantages

- High throughput
- Efficient for scientific computing
- Reduced instruction overhead
- Excellent pipeline utilization
- Large data parallelism
- High memory bandwidth utilization
- Ideal for matrix operations
- Good energy efficiency for data-parallel workloads

Vector Pipeline

Disadvantages

- Expensive hardware
- Complex compiler support
- Limited benefit for irregular algorithms
- Requires high memory bandwidth
- Poor performance on branch-heavy programs

Vector Pipeline

Applications

Scientific Computing

- Weather prediction
- Climate modeling
- Computational physics

Artificial Intelligence

- Neural network training
- Deep learning inference

Graphics

- Image processing
- Video processing
- Computer vision

Engineering

- Finite Element Analysis
- CAD/CAM
- Signal processing

Medical

- MRI reconstruction
- CT scan analysis